

ДИСЦИПЛИНА	Базы данных и анализ промышленных данных
ИНСТИТУТ	Институт перспективных технологий и индустриального программирования
КАФЕДРА	Кафедра индустриального программирования
ВИД УЧЕБНОГО МАТЕРИАЛА	Практическое занятие
ПРЕПОДАВАТЕЛЬ	Григорьева Анастасия Алексеевна
СЕМЕСТР	2 семестр, 2025 - 2026 гг.

Практическая работа 14

Работа с текстовыми данными.

Текст является одним из самых богатых, но и наиболее сложных для структурирования источников данных. В отличие от традиционных числовых таблиц, неструктурированный текст требует сложной трансформации в признаки, пригодные для анализа и машинного обучения.

Рассмотрим основные этапы работы с текстовыми данными.

1. Формирование корпуса и первичная предобработка

Первый шаг в работе с текстовыми данными – создание корпуса, то есть структурированной коллекции текстовых документов. Документ в данном случае представляет единицу информации, это может быть отзыв на товар, новость и т.д.

Первичная предобработка состоит из:

- ☐ приведения всех символов к нижнему регистру;
 - ☐ удаления HTML-тегов, URL-адресов и спецсимволов (знаков препинания, табуляций, переносов строк), которые не несут смысловой нагрузки, но создают избыточную размерность пространства признаков;
 - ☐ токенизации – разбиение текста на отдельные слова (токены).
- Важно отметить, что на этом этапе не следует удалять стоп-слова (предлоги, союзы, частицы), так как они необходимы для корректной работы последующих алгоритмов разметки.

2. Морфологический анализ и извлечение существительных

Под морфологическим анализом понимается автоматизированное решение двух взаимосвязанных задач: лемматизации (или стемминга) и частеречной разметки (POS-tagging). Лемматизация позволяет привести слова к нормальной форме, например, «стола», «столу», «столом» при лемматизации преобразуются в «стол», стемминг же отсекает суффиксы и окончания.

Второй компонент – частеречная разметка (Part-of-Speech tagging, POS-tagging). Это процесс присвоения каждому токenu (слову) метки, указывающей на его грамматическую категорию: существительное (NOUN), глагол (VERB), прилагательное (ADJ), местоимение (PRON), предлог (ADP) и так далее.

3. Построение признаков

В зависимости от решаемой задачи, объема данных и вычислительных ресурсов текстовые данные могут представляться различным образом, ниже представлены основные методы:

- ❑ Bag-of-Nouns (BoN), описывает документы как множество существительных и их частоту, игнорируя порядок слов;
- ❑ Bag-of-Words (BoW) в отличие от BoN хранит все слова и их частоту;
- ❑ n-граммы, представляет собой метод хранения фраз (в зависимости от n). Например, при n=2 биграмма будет представлять собой словосочетание «горячий кофе», «долгая доставка» и т.д
- ❑ TF-IDF, схож с BoW, но при этом вычисляет взвешенную частоту слов, тем самым выделяя более важные слова для конкретного документа.
- ❑ Векторные представления слов (Word Embeddings), которые учитывают контекст и создают плотные векторы (Word2Vec, FastText)

Рассмотрим пример обработки текстовых данных:

```
import pandas as pd
import re # Библиотека для работы с регулярными выражениями
import nltk # Библиотека для обработки текста
from nltk import word_tokenize # Токенизация
from nltk.corpus import stopwords # Стоп слова
import pymorphy3 # Лемматизация и морфологический анализ
from collections import Counter # Библиотека для подсчета частоты слов
from wordcloud import WordCloud # Визуализация текстовых данных
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

df = pd.read_csv('data.csv') # Загружаем датасет
https://www.kaggle.com/datasets/theovall/phonereviews

df = df.drop_duplicates() # Удаляем дубликаты
df.head()

  Review # Rating
0  3D Touch просто восхитительная 5
1  Отключается при температуре б 4
2  В Apple окончательно решили не 3
3  Постарался наиболее ёмко и ког 4
4  Достойный телефон. Пользовать 5

df = df[df['Rating']==5][:20000] # Для примера возьмем первые 20 тысяч
отзывов с рейтингом 5

df['Review'] = df['Review'].astype('str')
df['Review_clean'] = df['Review'].apply(lambda x: x.lower()) # Понижаем
регистр

df['Review_clean'] = df['Review_clean'].apply(lambda x: re.sub(r'^a-яА-
Яа-zA-Z0-9 ', '', x)) # С помощью регулярного выражения удаляем лишние
символы
```

```
# оставляем только буквы русского и латинского алфавитов, числа и пробелы
(^ означает отрицание)
```

```
df['Review_clean_token'] = df['Review_clean'].apply(lambda x: word_tokenize(x, language='russian')) # разбиваем строку на токены
```

```
df.head()
```

	Review	# Rating	Review...	Review_clean_token	...
0	3D Touch прс	5	3d touch прсто	['3d', 'touch', 'прсто', 'восхитительная', 'вещь', 'заряд', 'держит', 'целый', 'день', 'в', 'розовом', 'цвете', 'смотрится', 'оч	
4	Достойный т	5	достойный тел	['достойный', 'телефон', 'пользоваться', 'одно', 'удовольствие']	
5	6s gold 64gb	5	6s gold 64gb	['6s', 'gold', '64gb']	
6	Мой первый ай	5	мой первый ай	['мой', 'первый', 'айфон', 'скажу', 'честно', 'эпл', 'ранее', 'никогда', 'в', 'руках', 'не', 'держала', 'хотелось', 'попробоват	
7	Мне очень п	5	мне очень пон	['мне', 'очень', 'понравилась', 'эта', 'модель', 'вопервых', 'классный', 'дизайн', 'вовторых', 'круто', 'снимает', 'в', 'поме	

```
# Для лемматизации и выбора существительных напомним функцию
```

```
morph = pymorphy3.MorphAnalyzer()
```

```
def lemmatize_and_filter(tokens, pos_filter='NOUN'):
```

```
    result = []
```

```
    for token in tokens:
```

```
        if len(token) < 2: # Пропускаем короткие токены, такие как а, на,
        в и т.д.
```

```
            continue
```

```
        parsed = morph.parse(token)[0] # Морфологический анализ
```

```
        if parsed.tag.POS == pos_filter: # Проверяем на существительное
            result.append(parsed.normal_form)
```

```
    return result
```

```
df['Review_clean_noun'] = df['Review_clean_token'].apply(lambda x: lemmatize_and_filter(x))
```

```
df.head() # в итоге остались только существительные
```

	Review	#	Rating	Review...	Review...	Review_clean_noun	...
0	3D Touch прс		5	3d touch про	['3d', 'touch', 'пр	['вещь', 'заряд', 'день', 'цвет', 'телефон']	
4	Достойный т		5	достойный т	['достойный']	['телефон', 'удовольствие']	
5	6s gold 64gb		5	6s gold 64gb	['6s', 'gold', '6	[]	
6	Мой первый ай		5	мой первый ай	['мой', 'первы	['айфон', 'эппл', 'рука', 'андроид', 'дизайн', 'качество', 'сборка', 'высота', 'телефон', 'чехол', 'дисплей', 'мир',	
7	Мне очень п		5	мне очень п	['мне', 'очен	['модель', 'вопервое', 'дизайн', 'вовторое', 'помещение', 'счёт', 'взгляд', 'картинка']	

```
all_nouns = [noun for nouns_list in df['Review_clean_noun'] for noun in nouns_list] # Создаем список всех существительных
```

```
len(set(all_nouns)) # К-во уникальных существительных
```

24352

```
noun_freq = Counter(all_nouns) # Считаем количество существительных
```

```
print(noun_freq.most_common(10)) # В итоге получаем мешок существительных или BoN
```

```
# Визуализируем топ
```

```
top_nouns = noun_freq.most_common(15)
```

```
words, counts = zip(*top_nouns)
```

```
plt.figure(figsize=(10, 6))
```

```
plt.barh(words, counts, color='steelblue')
```

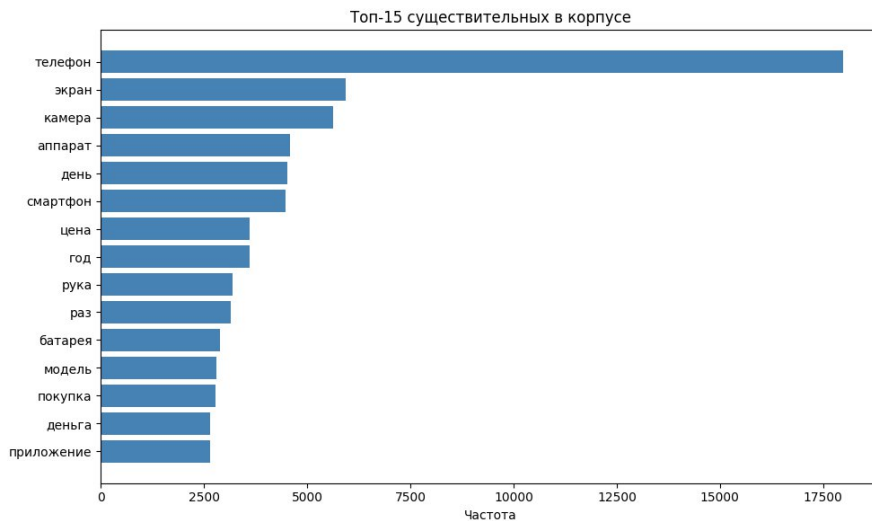
```
plt.xlabel('Частота')
```

```
plt.title('Топ-15 существительных в корпусе')
```

```
plt.gca().invert_yaxis()
```

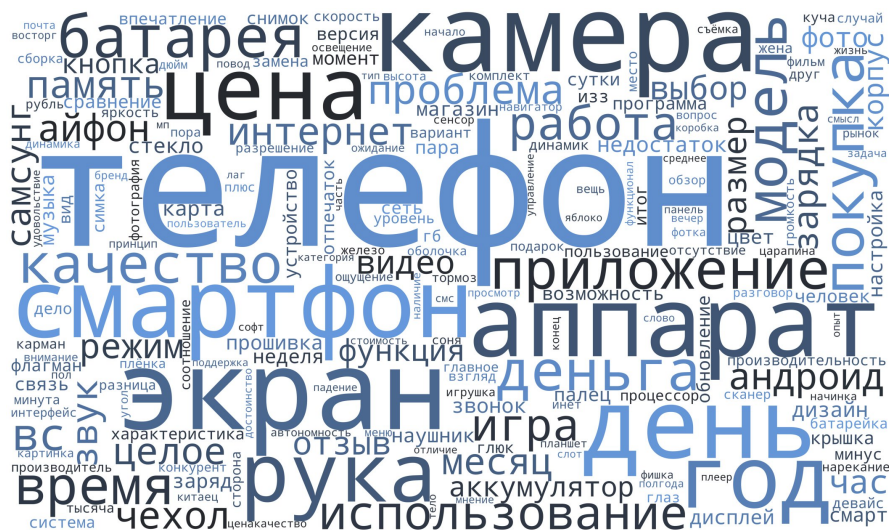
```
plt.tight_layout()
```

```
plt.show()
```



```
# Также для визуализации частотного анализа можно использовать облако
слов
docs = ' '.join(all_nouns) # Собираем все существительные в одну строку
stop_words = set(stopwords.words('russian')) # Задаем массив стоп-слов
wordCloud = WordCloud(width = 17000, height = 10000, # Строим облако
                      random_state=1, background_color='white',
                      colormap=sns.dark_palette("#69d", reverse=True,
as_cmap=True),
                      collocations=False, stopwords=stop_words).generate(docs)

plt.figure(figsize=(27,21))
plt.axis('off')
plt.imshow(wordCloud)
```



Задание 1 (4 балла)

1. Скачать данные по ссылке , либо спарсить интересующие должности и их описание с hh.ru (<https://www.kaggle.com/datasets/samoilovmikhail/partly-synthetic-dataset-oj-job-applications?select=jobs.csv>);
2. Выбрать 2-3 профессии (например, программист, аналитик, дата инженер);

3. Очистить описание вакансий от лишних символов, стоп-слов, удалить характерные для каждой вакансии слова (например «обязанности» и т.д.);
4. Провести частотный анализ описаний вакансий и требований для каждой профессии;
5. Выделить общие обязанности и требования для профессий, а также их различия.

Результаты представить в формате презентации.